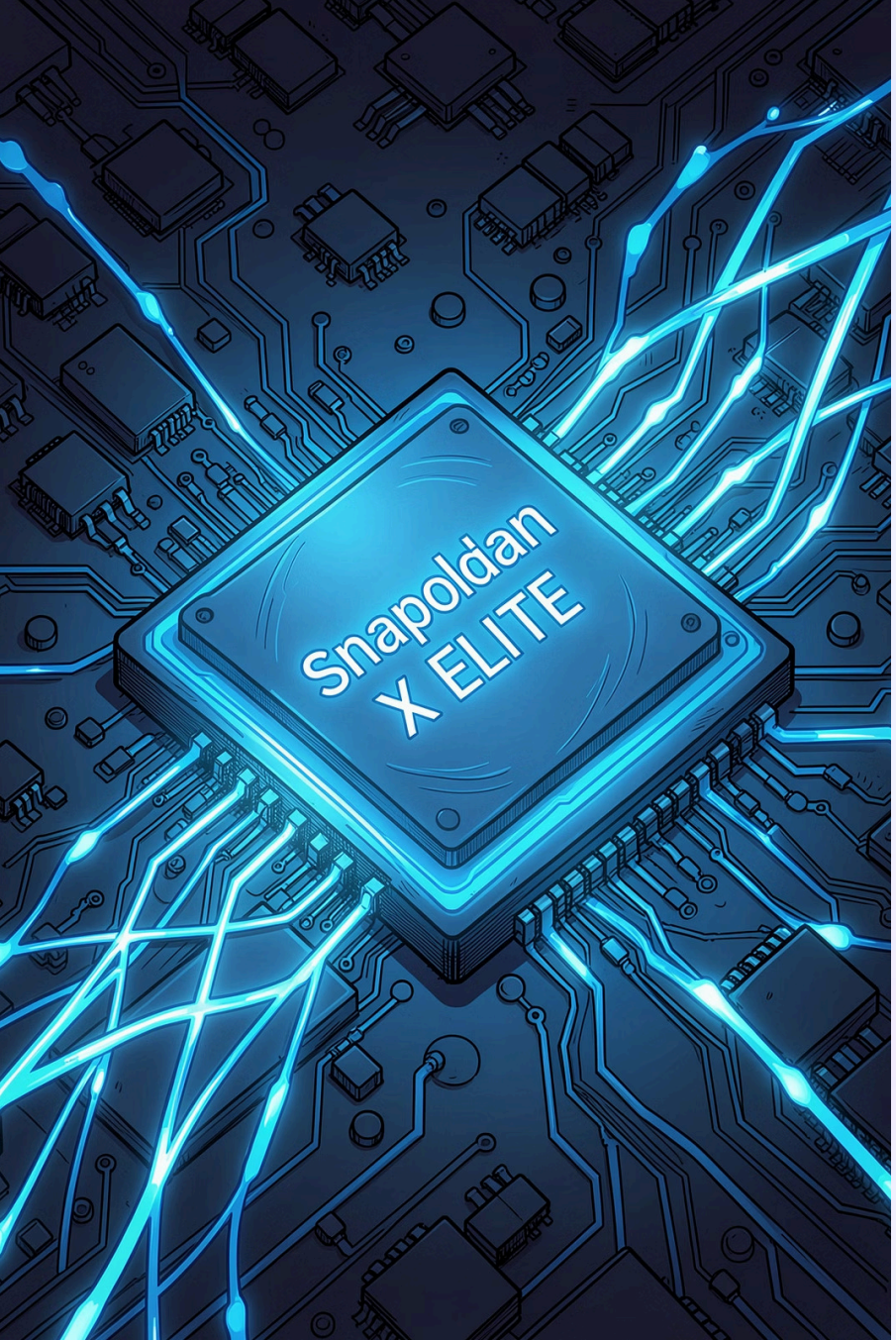
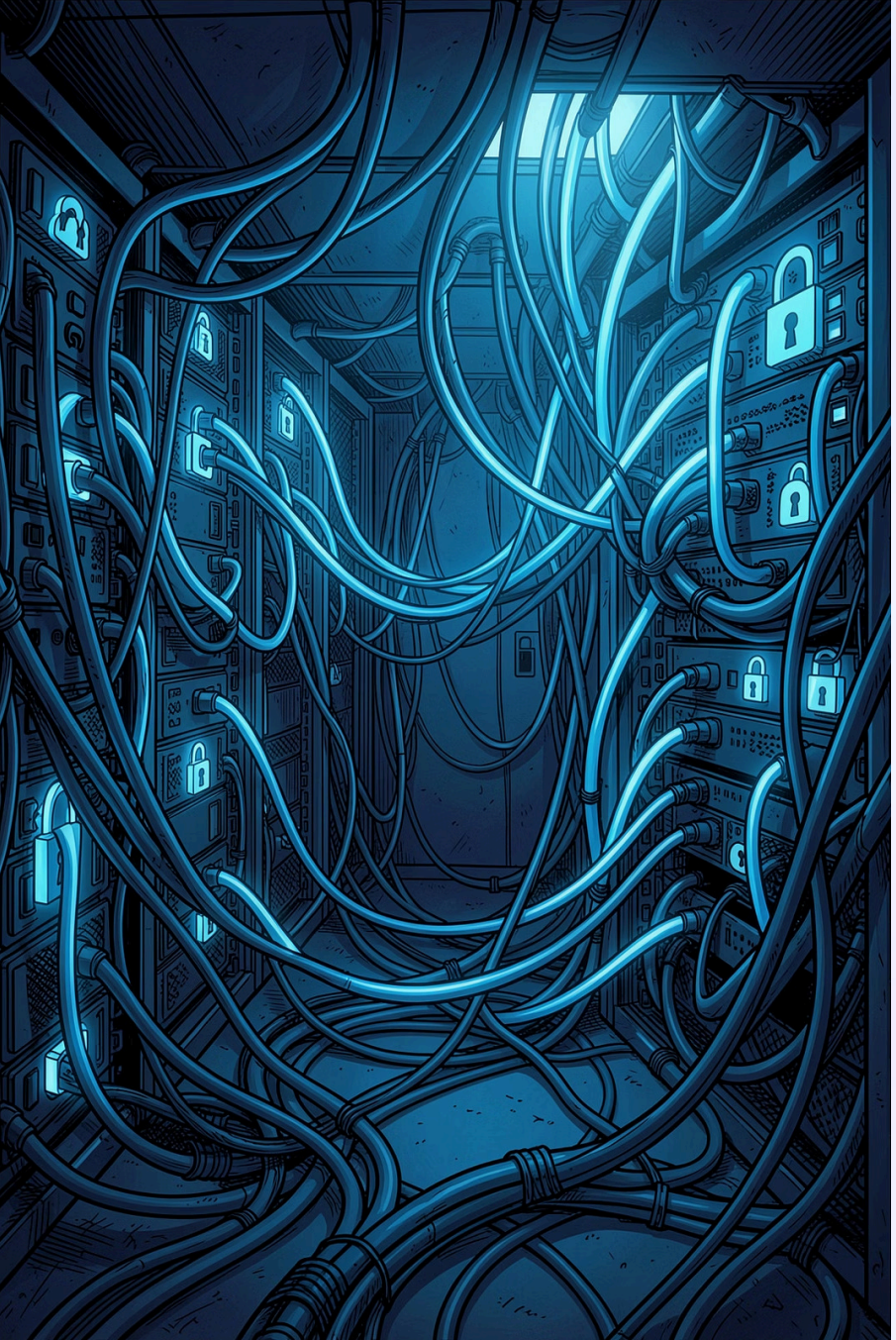


On-Device RAG System using Local LLMs

Built using Snapdragon X Elite — A privacy-preserving, offline-capable Retrieval Augmented Generation pipeline for mission-critical digital archives.

INTERNSHIP PROJECT | ENGINEERING R&D | QUALCOMM PLATFORM



PROBLEM

Why On-Device AI?

Privacy Risk

Cloud APIs transmit sensitive content to external servers — unacceptable for mission archives and confidential publications.

Offline Gap

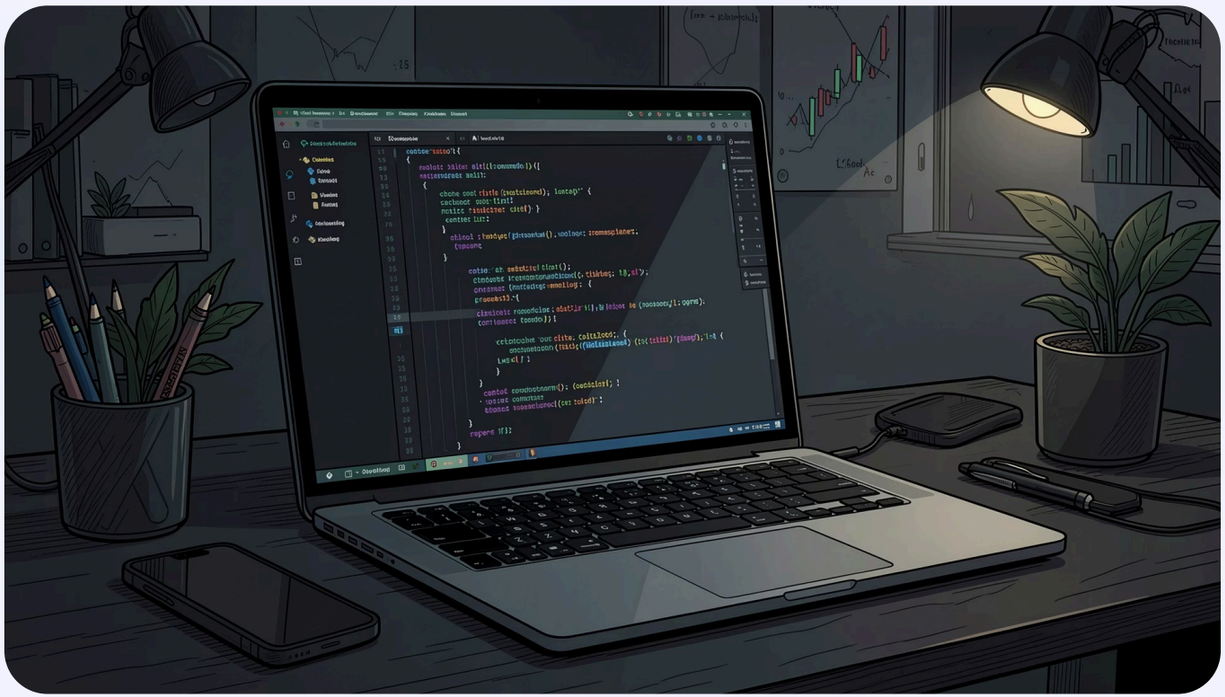
Cloud-only inference fails in low-connectivity or air-gapped environments where digital archives must remain accessible.

Latency & Cost

API round-trips add unpredictable latency. At scale, token costs become prohibitive for large document corpora.

GOALS

Project Objectives



Run LLM Locally

Deploy Qwen2.5 7B entirely on-device via Ollama — no external API calls.

Implement RAG Pipeline

Build end-to-end retrieval augmented generation with ChromaDB vector search.

Benchmark Performance

Measure TTFT, tokens/sec, RAM, and CPU usage on Snapdragon X Elite.

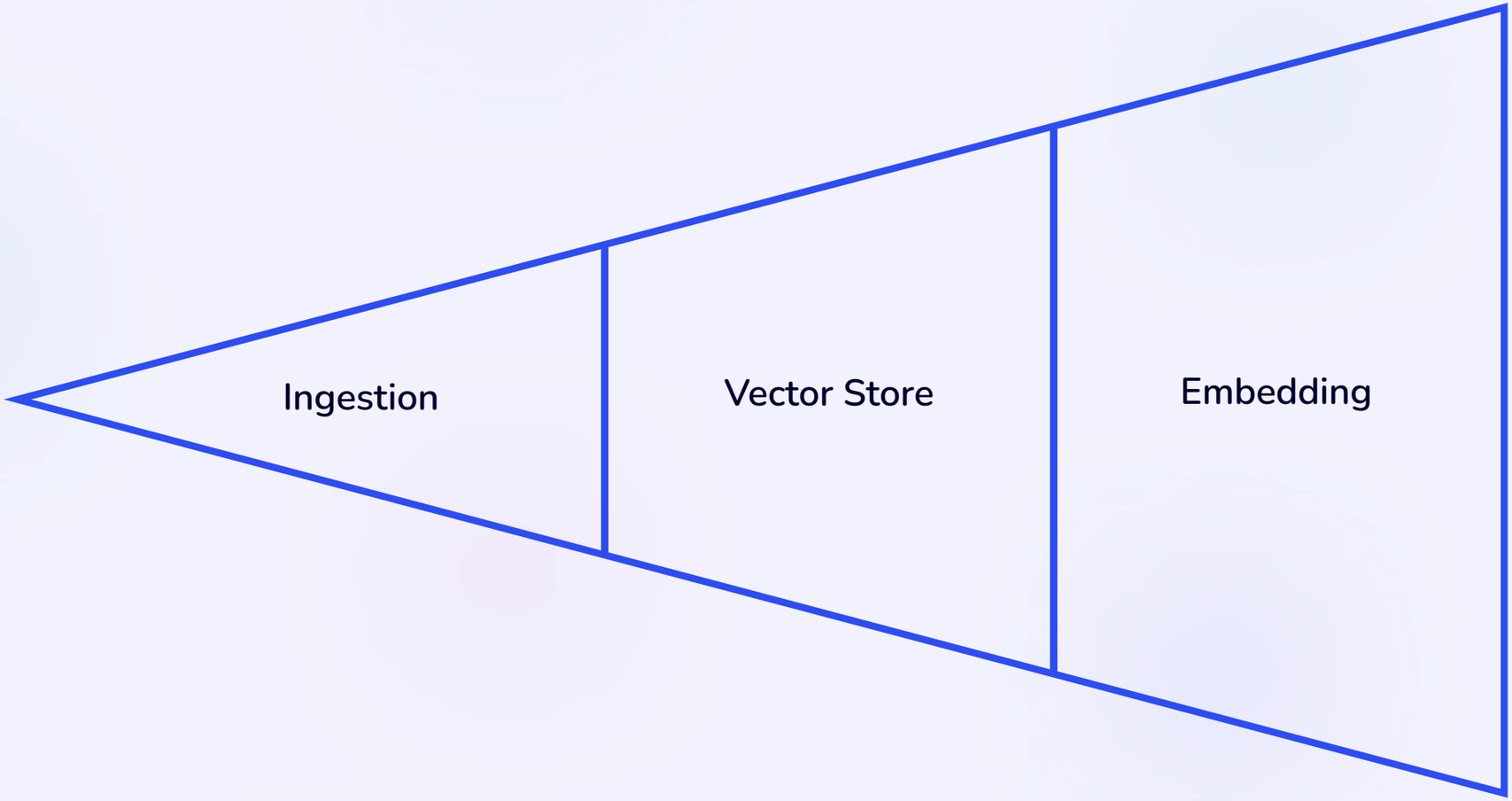
Semantic Retrieval

Enable accurate, context-aware answers over the Swami Vivekananda corpus.

CONCEPT

What is RAG?

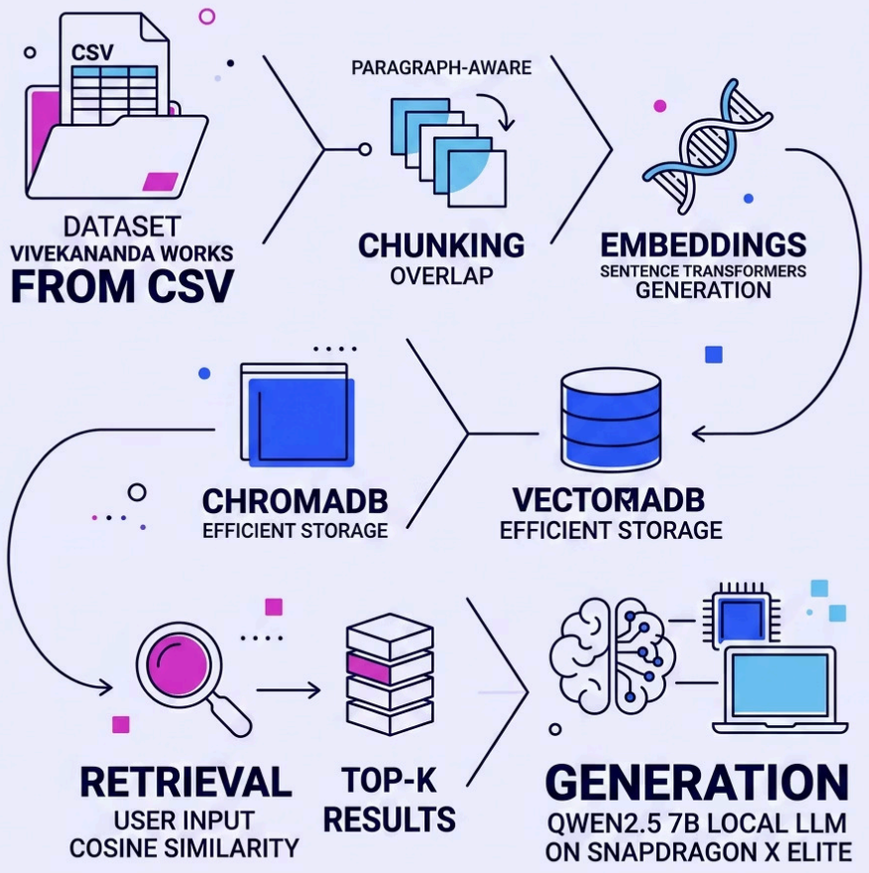
Retrieval Augmented Generation grounds LLM responses in verified source documents — reducing hallucinations and enabling domain-specific Q&A.



Each stage is fully local — from document ingestion through final token generation — ensuring data never leaves the device.

ARCHITECTURE

System Architecture



Hardware Platform

Snapdragon X Elite

ARM64 architecture with dedicated NPU for AI acceleration

32 GB RAM

Enables full 7B model + vector store in memory simultaneously

On-Device Inference

Zero network dependency; full privacy by design

STACK

Technologies & Dataset

Technology Stack

Python

Core pipeline logic

Ollama + Qwen2.5 7B

Local LLM runtime

ChromaDB

Vector similarity search

SentenceTransformers

Embedding generation

Pandas

CSV ingestion & preprocessing

Psutil

RAM & CPU monitoring

Dataset

Complete Works of Swami Vivekananda — ingested via CSV with local offline storage. Chosen for its dense philosophical content, making it an ideal testbed for semantic retrieval quality and multi-turn Q&A accuracy.

Semantic retrieval over this corpus demonstrates real-world RAG applicability for digital publications and mission archives.

IMPLEMENTATION

Chunking Strategy & Retrieval Pipeline

Chunking Evolution

1 Initial Approach

Fixed-size splits caused semantic fragmentation — context was lost mid-sentence.

2 Paragraph-Aware

Chunk boundaries aligned to natural paragraph breaks, preserving meaning.

3 Overlap-Based

Sliding window overlap ensures no context gap at chunk boundaries.

Retrieval Pipeline

01

Embed Query

SentenceTransformers encodes user input into a dense vector.

02

Cosine Similarity

ChromaDB computes similarity scores across all stored chunks.

03

TOP-K Selection

Top-N relevant chunks selected and assembled into context window.

04

LLM Generation

Qwen2.5 7B generates grounded, cited answer from retrieved context.

BENCHMARKS

Performance Metrics on Snapdragon X Elite

Key Observations

⚡ TTFT: 160-170s

Acceptable for offline local inference; dominated by model warm-up on ARM64.

📊 Throughput: 15–20 tok/s

Consistent streaming output after first token; suitable for interactive Q&A.

💾 RAM: ~19 GB

Full pipeline footprint — model + embeddings + ChromaDB — within 32 GB budget.

🔒 Zero Network Calls

100% offline operation confirmed across all benchmark runs.

LEARNINGS

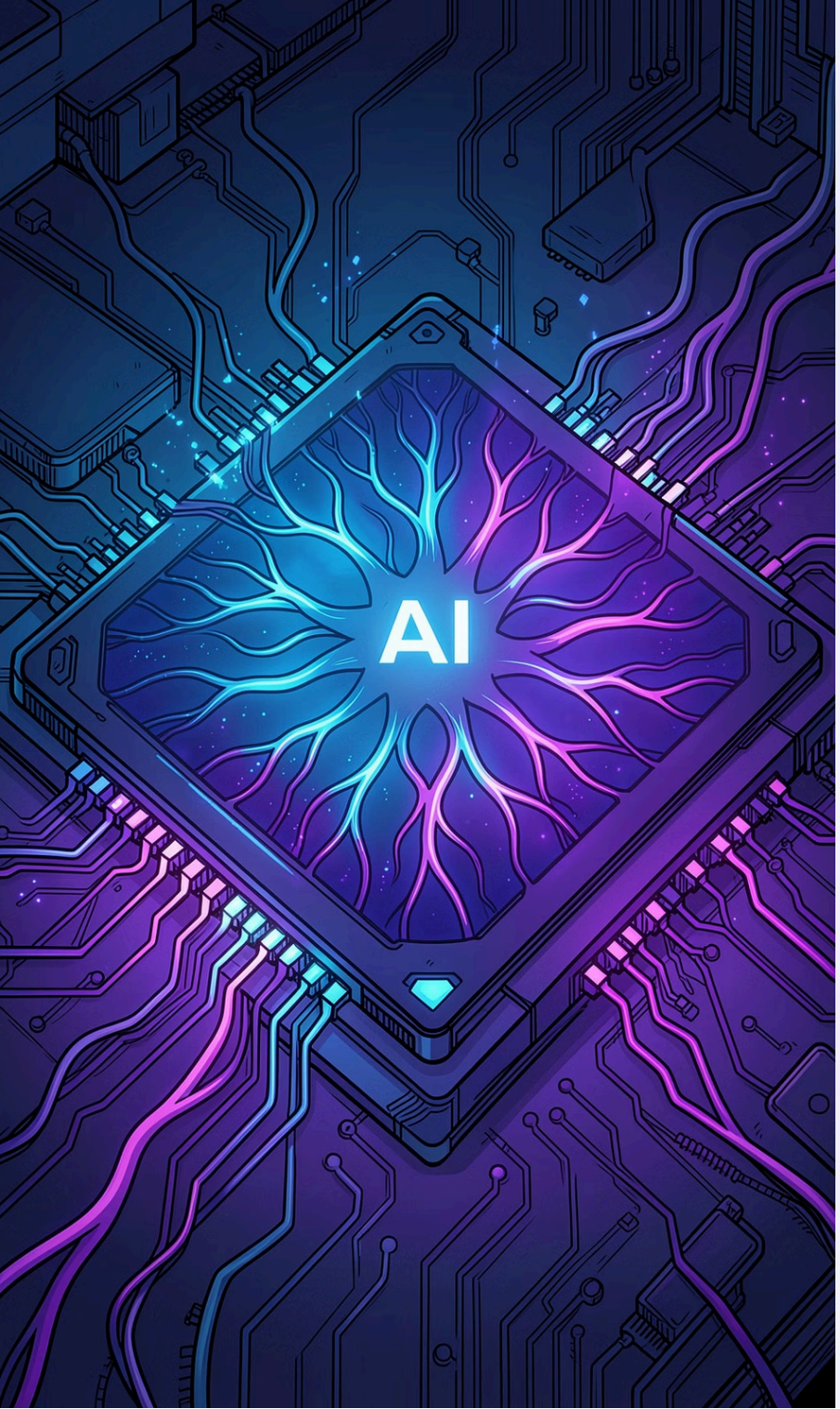
Challenges & Optimizations

⚠️ Challenges Faced

- **HuggingFace gated models** — auth token configuration required for Qwen download
- **ChromaDB batch limits** — large CSV ingestion hit insertion caps
- **ARM64 dependencies** — several Python wheels lacked native ARM builds
- **Poor initial retrieval** — naive chunking produced irrelevant context
- **Storage constraints** — model weights + vector store exceeded initial allocation

✅ Optimizations Applied

- **Paragraph-aware chunking** with overlap preserved semantic coherence
- **Increased TOP_K** from 3 → 5 for richer context assembly
- **Batch insertion** into ChromaDB replaced single-record writes
- **Offline pipeline** eliminated all runtime network dependencies
- **Context overlap** ensured no information loss at chunk boundaries



CONCLUSION

Looking Ahead

✅ Delivered

Successful local RAG deployment on Snapdragon X Elite — fully offline, privacy-preserving GenAI workflow demonstrated and benchmarked.

🚀 Next Steps

NPU acceleration, INT4 quantization, hybrid cloud/local routing, PDF-native ingestion, and streaming response optimization.

📈 Scalability

Architecture generalizes to any domain corpus — proven foundation for enterprise-grade on-device AI.

Questions? — Thank you for your time and feedback.